BlockChain Assignment 2 Report
Thato Phenyo Molete - 216038155
Github repository- Branch Assignment 2: https://github.com/phenyotmolete/BLCH9X2-Block_Chain/tree/assignment2

1 Overview: This assignment implements a single-block ledger unit for the simplified Bitcoin-like chain built in BLCH9X2. We briefly define a block as a structured record characterised by the following 3 features: (i) A block carries payload data, (ii) a block can be linked to its parent via a hash pointer, and (iii) a block stores a cryptographic commitment over its own material fields. The attached file, block.py, shows how I was able to implement a Block class. The file shows the documented canonical serialisation, a genesis block, a linked successor, and a tamper-evidence demonstration. I have tried to closely follow the materials and techniques established in Lecture 2 (sha256_string, GENESIS_PREVIOUS_HASH, create_genesis_block, create_linked_block, is_hash_valid).

2 The block: 2.1 Anatomy of the block: The block has 6 fields, with 5 entering the commitment but the hash being excluded. Functioning similar to an animal with 6 vital organs, the liver, heart, brain, lungs and pancreas being internal and the skin being external.
Fields: Index (integer height which tells you which number block the one we're looking at is along the chain, the genesis block is denoted by 0), timestamp (the recorded time of creation, preferred convention is of an integer Unix time stamp), transactions (the payload list- a list of simplified sender/recipient/amount dicts), previous_hash (the parent's stored hash), nonce (a PoW counter, unused at difficulty 0 but included for forward compatibility with the mining module), and hash (the stored commitment, never stored in its pre-image).

2.2 Canonical recipe: A hash commitment needs a unique byte string for identical logical content. The five material fields (which include everything except the hash itself) are collected by payload_for_hash() and serialised with:
raw = json.dumps(payload, sort_keys=True, separators=(",", ":"))
hash = sha256(raw.encode("utf-8")).hexdigest()
sort_keys=True removes insertion-order variance (a dict built as {b, a} and one built as {a, b} serialise identically). The compact separators remove whitespace variance between Python versions and platforms. Nested transaction dictionaries inherit the same stability because the whole payload passes through the same json.dumps call.
2.3 Genesis block: The first unique block. The well defined root of the chain, all later links are relative to this commitment. GENESIS_PREVIOUS_HASH = "0" * 64 , sixty-four ASCII zeros. This is our documented marker, not the hash of a real prior block. It is used consistently in code and in this report.
2.4 Chain linking: Setting previous hash of block i equal to the stored hash of block i-1, so the successor block cryptographically names its ancestor. When using create_linked_block() , we see it sets the child block's index to previous.index + 1 and its previous_hash to previous.hash, so the invariant previous_hash[i] == hash[i-1] holds by construction for every block built through the helper.

3. Results from execution:
Genesis block:

```
[1] Genesis block
    index: 0
    timestamp: 1700000000
    transactions: []
    previous_hash: 0000000000000000000000000000000000000000000000000000000000000000
    nonce: 0
    hash: 6b5ab8051e3b3b82c560233df8634f71d8debba962f8e2d28b898c012d5ff2f1
```

Canonicalisation:

In [16]:
```
# Show the exact canonicalisation used for the commitment
raw = canonical_dumps(genesis.payload_for_hash())
print("\n[2] Canonical JSON fed into the hash")
print(f"    {raw}")
print(f"    sha256(...) = {sha256_string(raw)}")
assert sha256_string(raw) == genesis.hash
```

```
[2] Canonical JSON fed into the hash
    {"index":0,"nonce":0,"previous_hash":"0000000000000000000000000000000000000000000000000000000000000000","timestamp":1700000000,"transactio
ns":[]}
    sha256(...) = 6b5ab8051e3b3b82c560233df8634f71d8debba962f8e2d28b898c012d5ff2f1
```

Block 1 (linked to the parent block 0):

In [17]:
```
# (c) Link a second block
sample_tx = [{"sender": "Alice", "recipient": "Bob", "amount": 10}]
block1 = create_linked_block(genesis, sample_tx, timestamp=1_700_000_100)
print("\n[3] Block 1 (linked to genesis)")
for key, value in block1.to_dict().items():
    print(f"    {key}: {value}")
print(f"    previous_hash == genesis.hash ? {block1.previous_hash == genesis.hash}")
assert block1.previous_hash == genesis.hash
assert is_hash_valid(block1)
```

```
[3] Block 1 (linked to genesis)
    index: 1
    timestamp: 1700000100
    transactions: [{'sender': 'Alice', 'recipient': 'Bob', 'amount': 10}]
    previous_hash: 6b5ab8051e3b3b82c560233df8634f71d8debba962f8e2d28b898c012d5ff2f1
    nonce: 0
    hash: e3e436fbd939f7d7094277fa2b1cd5b3d68dee86e58956606c82cac6d61a6b94
    previous_hash == genesis.hash ? True
```

Tamper evidence:

In [18]:
```
# (d) Tamper evidence demo
print("\n[4] Tamper evidence demo")
tampered = deepcopy(block1)
stored_before = tampered.hash
print(f"    stored hash before tamper : {stored_before}")

tampered.transactions[0]["amount"] = 999  # silent post-hoc edit
recomputed = tampered.compute_hash()
print(f"    recomputed after amount=999 (unchanged stored hash): {recomputed}")
print(f"    stored == recomputed?      {stored_before == recomputed}")
print(f"    is_hash_valid(tampered)?   {is_hash_valid(tampered)}")

assert stored_before != recomputed
assert is_hash_valid(tampered) is False
# original block1 must be untouched (deepcopy protected it)
assert is_hash_valid(block1) is True
assert block1.transactions[0]["amount"] == 10
print("\nAll checks passed: genesis built, block linked, tamper detected.")
```

```
[4] Tamper evidence demo
    stored hash before tamper : e3e436fbd939f7d7094277fa2b1cd5b3d68dee86e58956606c82cac6d61a6b94
    recomputed after amount=999 (unchanged stored hash): 0e6f0e3d584f47549b03172669b030741d33606fdbeb1adc0f5eb1dec7320328
    stored == recomputed?      False
    is_hash_valid(tampered)?   False

All checks passed: genesis built, block linked, tamper detected.
```

Tamper evidence demonstration:
The output above shows the the required demonstration. A copy of block1 is made with deepcopy so the transaction list is not shared by reference with the original.
Transactions[0]["amount"] is then changed from 10 to 999 after the block's hash was already stored.

If we try to recompute the commitment (compute_hash()) m we see that the hash we get no longer matches the stored hash, so is_hash_valid() correctly returns False. The original block1 is confirmed unaffected (amount still 10, is_hash_valid still True). From these results we see this has the effect of isolating the edit to the tampered copy.

Reflection:
For a financial ledger, tamper-evidence is guaranteed rather than tamper-prevention. A block's stored hash is a claim, and any peer can recompute and compare rather than trust a single operator's word.
This is directly important when we're looking at financial engineering practice. A permissioned bank ledger built on hash-linked blocks lets an external auditor recompute each block's commitment and check every previous_hash against its parent's stored hash. So if there is even one single-field edit (e.g. a payment amount that is changed silently) it fails a check, even without open proof-of-work. The nonce field is mostly unused or like an accessory in that permissioned bank ledger environment, since there is no competitive mining. I have retained the nonce here to ensure my work is consistent with the API with the mining module which we will be exploring at a later stage in the course. The canonicalisation discipline (sorted keys, fixed separators, integer timestamps) is important: an unstable serialisation would make two economically identical ledger states hash differently, breaking reproducibility and any automated verification built on top of it.

AI Use Declaration:
Tool: Claude (Anthropic). Use: implementation of block.py and main.py was drafted with AI assistance, closely following the lecture material and class demos presented in the Lecture 2. I have reviewed and can explain every function (payload_for_hash, compute_hash, create_genesis_block, create_linked_block, is_hash_valid) and the tamper-evidence demo, and made no changes to the canonicalisation scheme beyond what was learned in the lecture and textbook .